Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes

TD : Ingénierie Dirigée par les Modèles
Transformation par modèle (M2M) avec ATL

Des familles aux personnes, des classes aux tables : lire, compléter et écrire des modules ATL

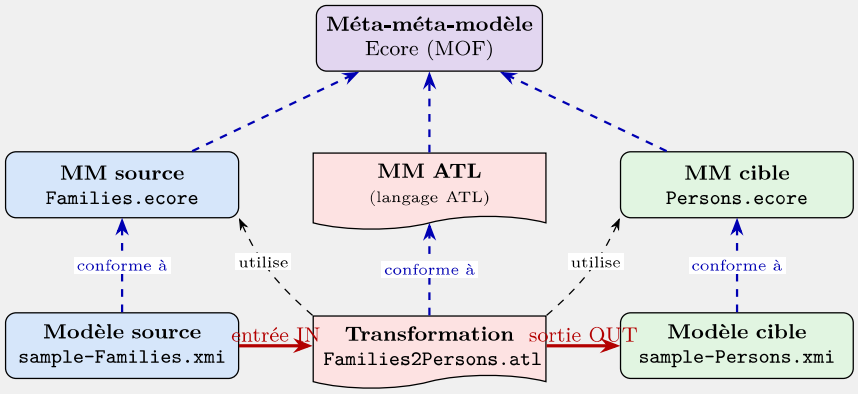
(Énoncé)

Pr.Aissam BERRAHOU
Année universitaire 2026–2027

Objectifs et consignes

1. Identifier les éléments d'une transformation M2M : **méta-modèles**, **modèles**, **module** ATL et relations de **conformité**.
2. **Lire** un module ATL : en-tête, **helpers**, **règles** (`from`, filtre, `to`, liaisons `<-`).
3. **Évaluer** des expressions OCL et **prédire** le modèle cible produit par une transformation.
4. **Écrire** des helpers et des règles ; **corriger** un module erroné.
5. Comprendre la **résolution automatique** des références (la **trace**).

Le TD suit le **fil rouge du cours** (Lecture 5) et du **tutoriel TP3 – Transformation par modélisation (ATL)** : la transformation `Families2Persons`. Un second exemple, `Class2Relational` (classes → tables), reprend la transformation PIM → PSM vue en Lecture 2. Les exercices se font **sur papier** ; le TP associé permet d'exécuter les mêmes modules dans Eclipse. L'aide-mémoire ATL (fin du cours) est autorisé.



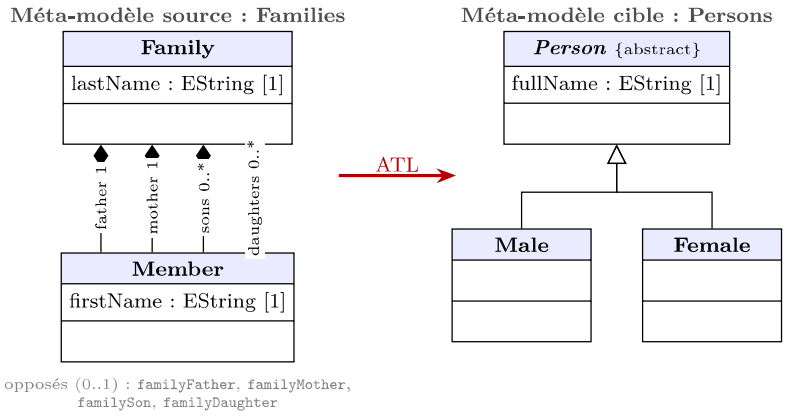
*La transformation est elle-même un **modèle** : tout est conforme à Ecore.*

Plan du TD :

Exercice	Thème	Notion principale
1	Vrai ou faux ?	Principe de la transformation M2M [Simple]
2	Les éléments d'une transformation	Méta-modèles, modèles, conformité [Simple]
3	Lire un module ATL	En-tête, helpers, règles [Simple]
4	Évaluer des expressions OCL	Navigation, `oclIsUndefined()`, collections [Simple]
5	Prédire le modèle cible	Filtres, ordre d'exécution [Moyen]
6	Écrire des helpers	Contexte, type, `if ...endif` [Moyen]
7	Compléter et modifier des règles	Motifs source et cible, liaisons [Moyen]
8	Trouver et corriger les erreurs	Syntaxe et sémantique ATL [Moyen]
9	Écrire `Class2Relational`	Trois règles et la trace [Avancé]
10	Regrouper les personnes par foyer	Résolution automatique, `flatten()` [Avancé]

Rappel : les méta-modèles et le modèle utilisés

Le méta-modèle source `Families` et le méta-modèle cible `Persons` du tutoriel. Dans `Families`, les références `father`, `mother`, `sons` et `daughters` sont des **compositions** ; chacune a une référence **opposée** `[0..1]` dans `Member` : `familyFather`, `familyMother`, `familySon`, `familyDaughter`. `Person` est **abstraite**.



Le modèle source `sample-Families.xmi` utilisé dans tout le TD :

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xmi:XMI xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI" xmlns="Families">
  <Family lastName="March">
    <father firstName="Jim"/>   <mother firstName="Cindy"/>
    <sons firstName="Brandon"/> <daughters firstName="Brenda"/>
  </Family>
  <Family lastName="Sailor">
    <father firstName="Peter"/> <mother firstName="Jackie"/>
    <sons firstName="David"/>   <sons firstName="Dylan"/>
    <daughters firstName="Kelly"/>
  </Family>
</xmi:XMI>
```

Le module `Families2Persons.atl` du tutoriel (numéros de lignes à gauche) :

```
1  module Families2Persons;
2  create OUT : Persons from IN : Families;
3
4  helper context Families!Member def: isFemale() : Boolean =
5      if not self.familyMother.oclIsUndefined() then true
6      else
7          if not self.familyDaughter.oclIsUndefined() then true
8          else false
9          endif
10     endif;
11
12 helper context Families!Member def: familyName : String =
13     if not self.familyFather.oclIsUndefined() then self.familyFather.lastName
14     else
15         if not self.familyMother.oclIsUndefined() then self.familyMother.lastName
16         else
17             if not self.familySon.oclIsUndefined() then self.familySon.lastName
18             else self.familyDaughter.lastName
19             endif
20         endif
21     endif;
22
23 rule Member2Male {
24     from s : Families!Member (not s.isFemale())
25     to   t : Persons!Male ( fullName <- s.firstName + ' ' + s.familyName )
26 }
27
28 rule Member2Female {
29     from s : Families!Member (s.isFemale())
30     to   t : Persons!Female ( fullName <- s.firstName + ' ' + s.familyName )
31 }
```

Exercice 1 : Vrai ou faux ? [Simple]

Répondre par **vrai** ou **faux** et **corriger** les affirmations fausses.

A1. Une transformation M2M produit du code source (Java, SQL…).

A2. Les règles ATL sont écrites au niveau des méta-modèles (M2) et s'appliquent à tout modèle conforme.

A3. `Families2Persons` est une transformation endogène.

A4. En ATL, le modèle source peut être modifié pendant la transformation.

A5. Dans `create OUT : Persons from IN : Families;`, `Persons` est le nom du modèle produit.

A6. Une matched rule s'applique automatiquement à chaque élément source qui satisfait son motif et son filtre.

A7. Le programmeur doit indiquer dans quel ordre les règles sont exécutées.

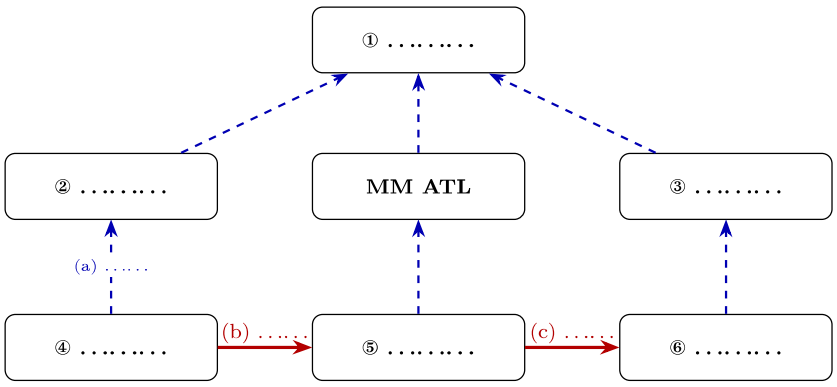
A8. Un helper peut modifier un attribut de l'élément sur lequel il est appelé.

A9. Une règle peut créer un élément de type `Persons!Person`.

A10. Les familles (`Family`) n'apparaissent pas dans le modèle cible.

Exercice 2 : Les éléments d'une transformation [Simple]

a) Compléter le schéma avec les noms suivants : `Ecore`, `Families.ecore`, `Persons.ecore`, `sample-Families.xmi`, `sample-Persons.xmi`, `Families2Persons.atl`. Compléter aussi les étiquettes (a), (b) et (c).



b) À quel niveau de la pyramide (M3, M2, M1) se trouve chacun des six éléments ?

c) Classer les transformations suivantes : exogène ou endogène ? verticale ou horizontale ? (i) `Families2Persons` ; (ii) une transformation qui renomme toutes les classes d'un diagramme UML en majuscules ; (iii) diagramme de classes (PIM) → schéma relationnel (PSM).

Exercice 3 : Lire un module ATL [Simple]

On considère le module `Families2Persons.atl` donné en rappel.

a) Quel est le nom du module ? Comment doit s'appeler le fichier ?

b) Quels sont les noms des modèles et des méta-modèles ? Lequel est lu, lequel est produit ?

c) Combien de helpers et de règles contient le module ? Sur quel type les helpers sont-ils définis ?

d) Quelle est la différence d'écriture entre `isFemale()` et `familyName` ? Comment appelle-t-on chacun dans une règle ?

e) Dans la règle `Member2Male`, identifier : le motif source, le filtre, le motif cible, la liaison.

f) Pourquoi les filtres des deux règles doivent-ils être **complémentaires** ?

Exercice 4 : Évaluer des expressions OCL [Simple]

Dans le modèle `sample-Families.xmi`, on note `f2` la famille Sailor, `peter`, `jackie`, `david`, `dylan` et `kelly` ses membres. Donner la valeur de chaque expression.

N°	Expression	Valeur
1	`kelly.firstName`	
2	`kelly.familyDaughter.lastName`	
3	`kelly.familySon.oclIsUndefined()`	
4	`dylan.isFemale()`	
5	`jackie.familyName`	
6	`f2.sons->size()`	
7	`f2.sons->collect(m | m.firstName)`	
8	`f2.daughters->isEmpty()`	
9	`Families!Member.allInstances()->size()`	
10	`Families!Member.allInstances()->select(m | m.isFemale())->size()`	
11	`peter.firstName + ' ' + peter.familyName`	
12	`if f2.sons->size() > 1 then 'grande' else 'petite' endif`	

Exercice 5 : Prédire le modèle cible [Moyen]

a) Donner le contenu du modèle cible `sample-Persons.xmi` produit par `Families2Persons` (type et `fullName` de chaque élément). Combien d'éléments `Male` et `Female` ?

b) On ajoute la famille suivante au modèle source. Quels éléments supplémentaires sont produits ?

```
<Family lastName="Benali">
  <father firstName="Ahmed"/>  <mother firstName="Fatima"/>
  <sons firstName="Yassine"/>  <daughters firstName="Salma"/> <daughters firstName="Nour"/>
</Family>
```

c) Que produit la transformation (sur le modèle de départ) si l'on **supprime** la règle `Member2Female` ?

d) Et si l'on remplace le filtre de `Member2Male` par `(true)` (sans toucher à `Member2Female`) ?

e) Les trois phases du moteur ATL sont : correspondance, création, initialisation. Décrire ce qui se passe pour **Cindy** lors de chacune.

Exercice 6 : Écrire des helpers [Moyen]

Écrire les helpers suivants (préciser s'il s'agit d'un helper attribut ou opération) :

a) `isParent : Boolean` sur `Member` : vrai pour un père ou une mère.

b) `nbChildren : Integer` sur `Family` : le nombre d'enfants (fils et filles). Valeur pour March ?

c) `family : Families!Family` sur `Member` : la famille du membre, quel que soit son rôle.

d) Réécrire `familyName` en **une ligne** grâce au helper `family`.

e) `role : String` sur `Member` : `'father'`, `'mother'`, `'son'` ou `'daughter'`.

f) `hasSon(name : String) : Boolean` sur `Family` : vrai si la famille a un fils de ce prénom. Valeur de `f2.hasSon('Dylan')` ?

Exercice 7 : Compléter et modifier des règles [Moyen]

a) Compléter les trous ① à ⑤ de la règle suivante :

```
rule Member2Female {
    ① .........
        s : ② ......... ( ③ ......... )
    to
        t : ④ ......... (
            fullName ⑤ ......... s.firstName + ' ' + s.familyName
        )
}
```

b) Modifier les deux règles pour que le nom complet soit de la forme `'MARCH, Jim'` (nom en majuscules, virgule, prénom). Que devient Kelly Sailor ?

c) On ajoute à la classe `Person` un attribut `role : EString`. Modifier `Member2Male` pour le remplir avec le helper `role` de l'exercice 6.

d) On veut maintenant **trois** types de personnes : `Father`, `Mother` et `Child`, sous-classes de `Person`. Écrire les filtres des trois règles `Member2Father`, `Member2Mother` et `Member2Child`. Vérifier qu'ils sont complémentaires.

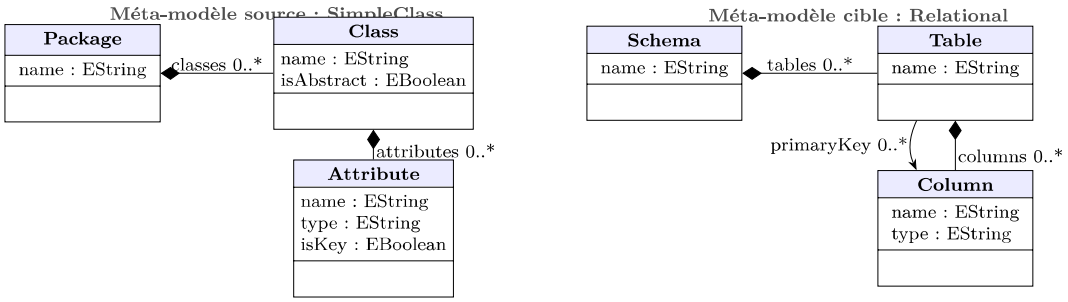
Exercice 8 : Trouver et corriger les erreurs [Moyen]

Le module suivant, enregistré dans le fichier `Families2Persons.atl`, contient **8 erreurs**. Pour chacune : donner la ligne, expliquer l'erreur et la corriger.

```
1  module Family2Person;
2  create OUT : Persons from IN : Families
3
4  helper context Families!Member def: isFemale() : Boolean =
5      if not self.familyMother.oclIsUndefined() then true
6      else not self.familyDaughter.oclIsUndefined();
7
8  rule Member2Male {
9      from s : Families!Member
10     to   t : Persons!Person (
11             fullName = s.firstName + " " + s.familyName )
12 }
13
14 rule Member2Female {
15     from s : Families!Member (s.isFemale())
16     to   t : Persons!Female (
17             fullName <- s.firstName + ' ' s.familyName )
18 }
```

Exercice 9 : Écrire la transformation Class2Relational [Avancé]

On veut transformer un modèle de classes simplifié en un schéma relationnel (transformation PIM → PSM de la Lecture 2). Les méta-modèles sont :



Règles de traduction : un `Package` donne un `Schema` de même nom ; une `Class` **non abstraite** donne une `Table` dont le nom est en **majuscules** ; un `Attribute` donne une `Column` ; le type est traduit ainsi : `String` → `VARCHAR(100)`, `Integer` → `INT`, `Boolean` → `BOOLEAN`, `Date` → `DATE`, autre → `VARCHAR(255)` ; la clé primaire d'une table est formée des colonnes issues des attributs `isKey = true`.

a) Écrire l'en-tête du module `Class2Relational`.

b) Écrire un helper `toSQL() : String` de contexte `String`.

c) Écrire les trois règles `Package2Schema`, `Class2Table` et `Attribute2Column`.

d) Dans `Package2Schema`, la liaison `tables <- p.classes` affecte des `Class` à une référence qui attend des `Table`. Pourquoi est-ce correct ?

e) Donner le modèle cible obtenu pour le paquetage `library` contenant `Book` (isbn : String, title : String, pages : Integer), `Member` (memberId : Integer, name : String, email : String) et `Loan` (loanDate : Date, returned : Boolean).

f) Une classe abstraite `Document` possède un attribut `ref : String`. Que devient cet attribut ? Proposer une correction.

Exercice 10 : Regrouper les personnes par foyer [Avancé]

On ajoute au méta-modèle `Persons` une classe `Household` (attribut `name : EString`, composition `members : Person [0..*]`).

a) Écrire une règle `Family2Household` qui crée un foyer par famille, de même nom, contenant toutes les personnes issues des membres de la famille.

b) Les règles `Member2Male` et `Member2Female` sont-elles à modifier ? Pourquoi ?

c) Combien d'éléments **racines** contient alors le modèle cible pour `sample-Families.xmi` ? Dessiner le résultat pour la famille March.

d) Peut-on écrire la transformation **inverse** `Persons2Families` (à partir de l'ancien méta-modèle Persons, sans foyers) ? Quelles informations manquent ?